

Instant Insanity on 12 qubits

Half the qubits, no penalty terms, and a 44% hit rate on AWS SV1 against a 0.146% random baseline.

Daksh Goswami · Rueben · Ishal

Qiskit Fall Fest Connecticut 2026 · qBraid challenge · UConn Quantum Computing

A tiny puzzle with a brutal search space

331,776

ways to stack four cubes

8

that actually work

< 1 s

for classical brute force

Every cube shows one of three opposite-face pairs on the front/back axis and a different pair on left/right, so each cube has exactly 6 legal choices. A tower is solved when all four sides show all four colours.

Brute force already wins at N = 4.
We say so on every slide that reports a number. What the quantum route buys is a method that never enumerates — and a cost curve we can measure as the puzzle grows.

Make every bitstring legal, and the penalties disappear

One-hot (starter)

- 6 qubits per cube → 24 for four cubes
- most bitstrings are illegal
- penalties for "one pair per axis"
- penalty for "a pair cannot serve both axes"
- XY mixer + W-state preparation

Compact (ours)

- 3 qubits per cube → 12 for four cubes
- all 8 states map onto the 6 legal choices
- no bitstring is ever illegal
- no penalty terms, nothing to tune
- plain X mixer is enough

24 → 12 qubits

cost = Σ over axes and colours of $(d_k - 2)^2$

zero exactly for a solved tower



We checked the model before we trusted a single shot

1

Hamiltonian = cost

On every one of the 4,096 bitstrings. Difference exactly 0.0, not a tolerance.

2

Two cost implementations

A vectorised one and a plain-Python one written straight from the definition.

3

Decoder, both directions

No zero-cost bitstring fails the starter's solves(); no positive-cost one passes it.

4

Fast simulator = Qiskit

The hand-written statevector used in the angle search agrees to 1e-15.

Every success rate we report is the fraction of shots that pass the classical checker — never the fraction below an energy threshold.

The state-to-choice map is a free 2.6× saving

Two of the eight states are duplicates. Which ones is ours to choose, and it changes the Walsh expansion of the cost — so it changes how many Z strings the Hamiltonian has and how long they are. We searched all 720 × 36 maps and scored each by the CX gates one cost layer needs. Same Hamiltonian, same answers, cheaper circuit.

1216

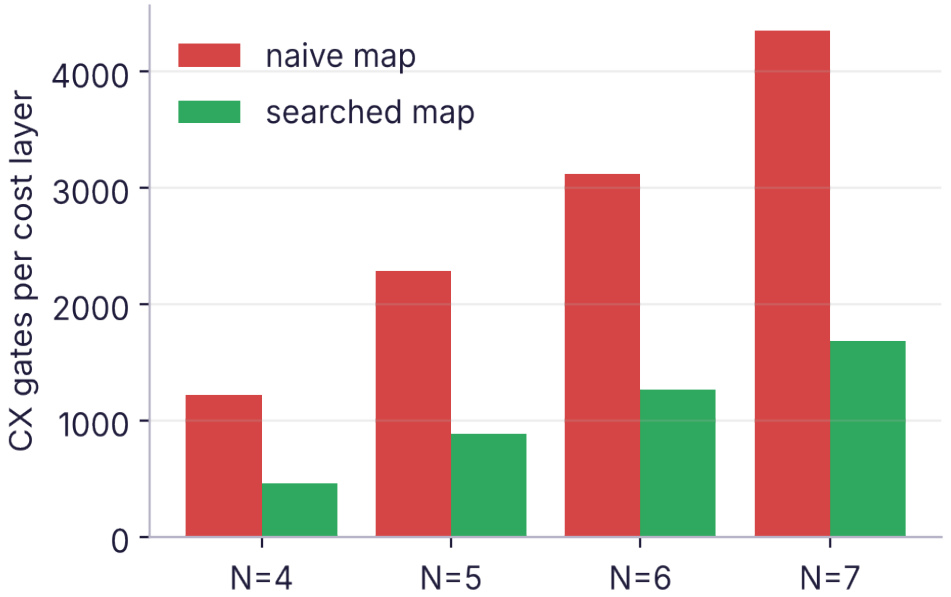
CX per layer, naive map

460

CX per layer, searched

6 → 4

longest Z string



The search is pure classical preprocessing: it runs once, costs seconds, and the circuit it produces is provably the same Hamiltonian.

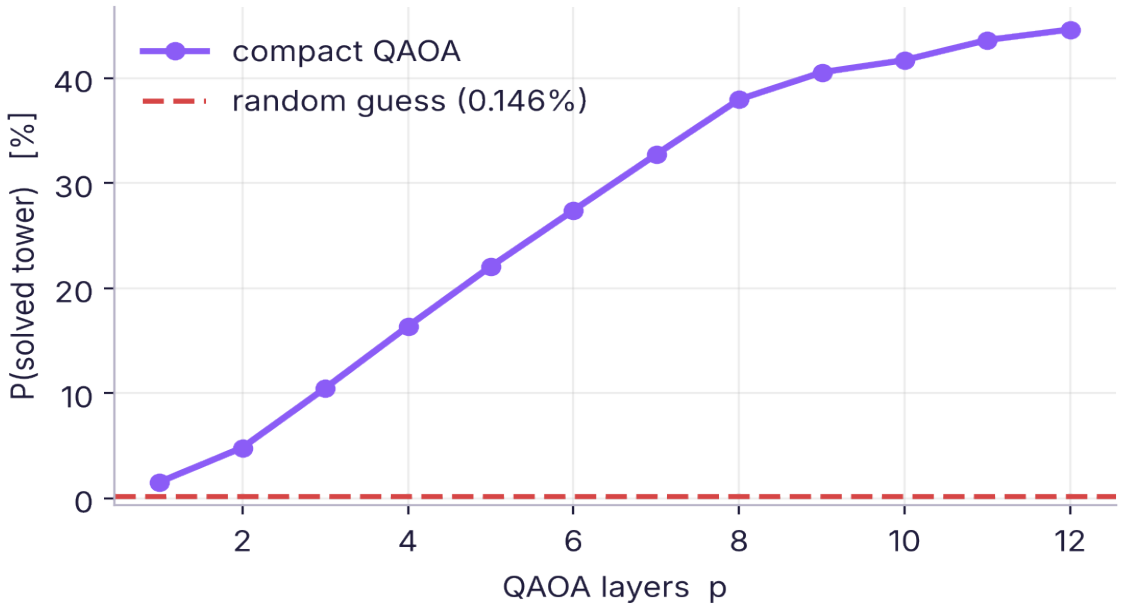
How we optimised mattered more than the circuit

CVaR instead of the mean

Minimising the average cost is not the same as maximising P(solved): the average is dominated by the bulk of bad bitstrings. CVaR averages only the best 30%, so the optimiser chases the low-cost tail where the answers are.

Layer-by-layer growth

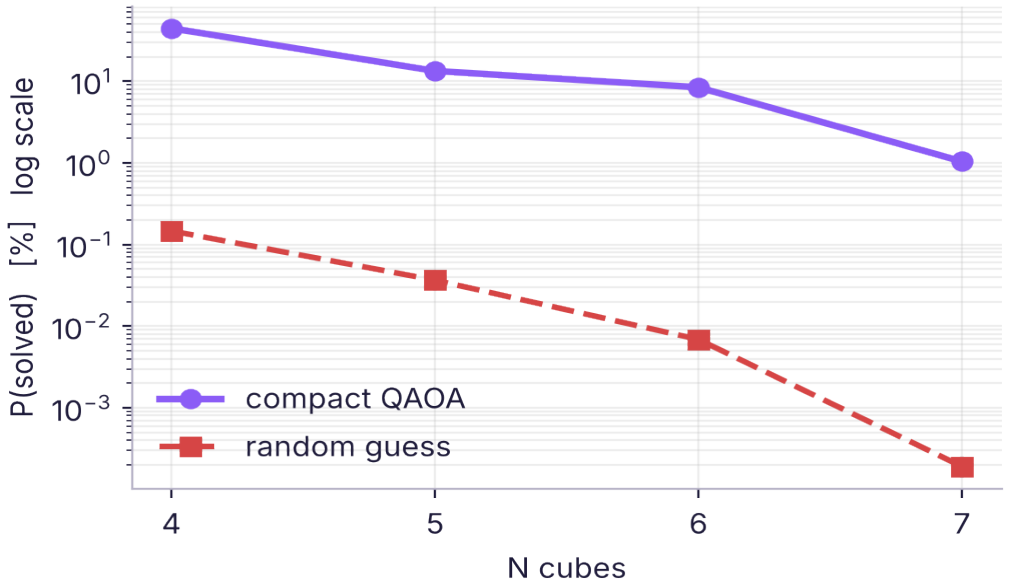
Optimising 2p angles from a random start stalls. Instead solve p = 1, interpolate those angles into a p + 1 schedule, re-optimise. Every layer starts from a good guess.



Same Hamiltonian, same hardware: ~2% → 44.6%

Results, and how they scale with N

N	Qubits	CX/layer	p	P(solved)	Random	Lift
4	12	460	12	44.20%	0.1465%	×302
5	15	890	10	13.37%	0.0366%	×365
6	18	1264	10	8.43%	0.0069%	×1227
7	21	1686	8	1.04%	0.0002%	×5457



N = 4 ran on AWS SV1: 884 of 2000 shots = 44.20% measured, against 44.62% predicted. Inside sampling error.
The lift over random grows with N, but the absolute success rate falls, because the solution fraction shrinks faster than the algorithm gains. **We report both, not only the flattering one.**

Puzzles with N > 4 from the starter's make_puzzle(n, seed): N=5 seed 16, N=6 seed 86, N=7 seed 68. Every N re-verified against the classical cost, and its alias map re-searched, before any angle search.

What we are not claiming

No quantum speedup. Brute force solves the four-cube puzzle in a fraction of a second, far faster than building, optimising and sampling these circuits. The honest comparison is against random guessing inside the same encoding, and that is what every table reports. The general N-cube problem is NP-complete, and nothing here changes that.

Next. The cost layer is the bottleneck, and most of it is weight-3 and weight-4 Z strings. Order them to share CX ladders between neighbouring terms, and warm-start the angles for N + 1 from the optimised schedule at N — the layerwise growth already hints it transfers.

